Light Fidelity (LiFi) using Raspberry Pi

EE1003-2026 Project Report

Aditya Appana -EE25BTECH11004 and Suraj N-EE25BTECH11056

1. Introduction

Optical wireless communication — commonly referred to as **LiFi** (Light Fidelity) — offers a high-security, RF-interference-free medium for short-range data transfer. Naturally confined to the physical space illuminated by the source, it presents a low-cost entry point for experimental communication research.

This report documents the iterative development of a *simplex* LiFi data link between two Raspberry Pi single-board computers, using an infrared (IR) LED and a reverse-biased photodiode as the physical medium. Beginning from first-principles hardware diagnostics, the project progressed through multiple protocol revisions, achieving stable transmission of raw text at **200 bps**, before encountering the hard architectural limits of pure-Python bit-banging during experimental audio transfer.

The central engineering tension throughout the project was reconciling the real-time demands of optical bit-level timing with the inherent scheduling jitter of a general-purpose Linux OS — a constraint that motivated every major design decision presented here.

2. Hardware Architecture

To eliminate analogue noise and avoid the need for operational amplifiers, the physical layer was reduced to a bare-minimum digital logic circuit. The design evolved in parallel with software capabilities.

2.1 Transmitter Circuit (TX)

The transmitter employs an NPN digital switch topology. A **BC547** BJT allows the 3.3 V GPIO logic of the Raspberry Pi to drive the IR LED from the higher-current 5 V rail, maximising photonic output and transmission range.

Table 1. Transmitter Circuit Wiring

Component	Connects To	Detail
Pi Pin 12 (GPIO 18)	BC547 Base	2 kΩ resistor
Pi Pin 2 (5 V)	IR LED Anode	100 Ω resistor
IR LED Cathode	BC547 Collector	Direct wire
BC547 Emitter	Pi Pin 6 (GND)	Direct wire

2.2 Receiver Circuit — Standard (RX-Standard)

For speeds up to 200 bps, a minimal-component receiver was used. A reverse-biased photodiode is tied to a 10 kΩ pull-up resistor. In ambient darkness, the GPIO pin reads a logic HIGH (3.3 V). Under IR illumination the photodiode becomes conductive, pulling the line to logic LOW (0 V) — a sharp digital swing polled continuously by the Python receiver.

Table 2. Standard Receiver Wiring (200 bps)

Component	Connects To	Detail
Pi Pin 1 (3.3 V)	Photodiode Cathode	10 kΩ pull-up
Photodiode Cathode	Pi Pin 16 (GPIO 23)	Signal node
Photodiode Anode	Pi Pin 6 (GND)	Direct wire

2.3 Receiver Circuit — UART Attempt (RX-UART)

To target 115,200 bps using the Pi's hardware UART silicon, a two-transistor logic inverter was constructed. Q1 acted as a light-sensitive amplifier; Q2 inverted the output to satisfy the UART idle-high convention. This design was ultimately sidelined: without an analogue front-end (transimpedance amplifier, comparator), bare photodiodes are insufficiently immune to ambient light at the sub-microsecond timing resolution UART demands. The attempt nonetheless confirmed the necessity of an analogue signal-conditioning stage for high-speed optical links.

3. Software Evolution and Protocols

The software layer underwent four major revisions, each motivated by a concrete performance bottleneck or reliability failure.

3.1 Phase 1 — Basic Bit-Banging (20 bps)

The first protocol implemented a standard **8-N-1** asynchronous serial frame. Each ASCII character is converted to its decimal value via Python's `ord()` function, then bit-shifted into eight discrete HIGH/LOW pulses bracketed by a start bit and a stop bit.

The pivotal design choice was scheduling every bit transition using *absolute* wall-clock timestamps (`time.time()`) rather than successive `time.sleep()` calls. Accumulated sleep-call error causes unbounded timing drift; absolute timestamps confine each error to a single bit period, yielding a stable, drift-free baseline of **20 bps**.

3.2 Phase 2 — High-Speed Optimisation (200 bps)

Profiling the Phase 1 receiver identified two I/O bottlenecks:

- Per-character `print()` calls introducing unpredictable blocking delays.
- Synchronous file writes after every received byte.

Both were resolved by batching: terminal updates throttled to every 50 characters; file writes deferred similarly. Reducing the bit period to $t_{\text{bit}} = 0.005\,\text{s}$ yielded a **10**× throughput gain to **200 bps** with no missed frames. The transmitter signals end-of-file with a null byte (`0x00`), prompting the receiver to safely close and save the output file. This proved exceptionally reliable for payloads ranging from short strings to multi-paragraph text documents.

3.3 Phase 3 — Audio Transmission via Base64 Encoding

Transmitting binary `.wav` files directly over the ASCII protocol failed: non-printable bytes (< 32 or > 126) crashed the receiver's parsing logic. The solution was a transparent encoding layer:

Encoding pipeline:

`.wav` binary $\xrightarrow{\text{base64}}$ safe ASCII $\xrightarrow{\text{LiFi}}$ decode $\rightarrow$ `.wav`

Base64 maps every 3 binary bytes to 4 printable characters, confining the alphabet to `[A-Za-z0-9+/=]` at a $\approx 33\%$ payload overhead. While this guaranteed protocol integrity, the phase ultimately exposed two hard limits described in Section **??**.

3.4 Phase 4 — Real-Time Non-Volatile Persistence

Long transmissions introduced the risk of total data loss from a transient interruption (a passing shadow, mechanical vibration). The receiver was upgraded with periodic forced disk commits:

```python
if chars_received % 5 == 0:
    f.flush()
    os.fsync(f.fileno())
```

Listing 1. Forced kernel bypass to physical storage

`os.fsync()` bypasses the kernel page cache, writing buffered data directly to the SD card. At most 5 characters are lost in an unrecoverable failure — a critical fault-tolerance measure for slow, long-duration transfers.

4. Bandwidth Limits: Audio Transmission Failure

Following the success of text transfer, the Phase 3 audio experiment was **deemed a failure** due to two compounding constraints.

4.1 Prohibitive Transmission Times

At 200 bps the effective throughput is ≈ 25 bytes per second. A minimal audio file — 0.5 s at 8 kHz, 8-bit mono — occupies roughly 8 KB before Base64 expansion.

Estimated transmission times at 200 bps:

1-second audio clip $\approx$ **5–7 minutes**
3-minute song $\approx$ **∼30 hours**

This rendered the system practically unusable for dynamic media, even at the protocol's maximum stable speed.

4.2 Catastrophic Data Corruption from OS Jitter

The extended transmission windows exposed the fundamental flaw of pure-Python GPIO bit-banging. Over a 5-minute window, the Linux kernel inevitably pre-empts the Python receiver for fractions of a millisecond to service background tasks (Wi-Fi polling, timer interrupts, filesystem housekeeping). During each pre-emption, an incoming light pulse is missed.

In ASCII text, a dropped bit corrupts one character — a tolerable typo. In a Base64-encoded audio stream, a single bit-slip permanently desynchronises the entire decoding sequence:

- All subsequent characters are shifted by one position.
- The resulting `.wav` files were either entirely unreadable by the OS or produced severe corrupted static.

Post-mortem analysis of live-saved Base64 text strings (Section **??**) confirmed that bit-slips occurred randomly throughout every transmission, even under optimal optical alignment and with ambient light fully excluded by collimation tubes. The conclusion was unambiguous: eradicating software jitter requires migrating to a hardware UART chip or DMA-capable C libraries — both beyond the scope of a pure-Python GPIO implementation.

5. Debugging and Alignment Methodology

Physical-layer alignment and electrical verification proved to be the most demanding aspects of the project. The following procedures were formalised through iterative experimentation.

5.1 Electrical Diagnostics

- **Multimeter Logic Probing.** The RX signal node was monitored to verify: (a) a firm idle-high of $\approx 3.3\,\mathrm{V}$ in darkness, and (b) a clean drop to $\approx 0\,\mathrm{V}$ under direct illumination, satisfying standard TTL thresholds.
- **Pull-the-Plug Test.** Temporarily removing the photodiode from the circuit confirmed whether a stuck-HIGH condition was caused by a wiring fault or ambient light saturation.

5.2 Optical Alignment Techniques

1. **Collimation Tubes.** Opaque cylinders fitted over the IR LED and photodiode restricted each component's field of view, suppressing 100 Hz fluorescent-light flicker from the received data stream.

6. Results Summary

Table **??** summarises the outcome of each development phase.

Table 3. Performance Summary Across Development Phases

Phase	Speed	Key Outcome	Status
1	20 bps	Stable 8-N-1 bit-banging; drift-free absolute timestamps	**Pass**
2	200 bps	10× gain via I/O batching; reliable text/file transfer	**Pass**
3	200 bps	Base64 audio encoding; protocol integrity achieved	**Partial**
4	200 bps	Real-time fsync persistence; OS jitter confirmed fatal for audio	**Fail**

7. Conclusion

This project demonstrated that a reliable, low-cost simplex LiFi network can be engineered from commodity discrete components and pure Python, provided the payload is strictly limited to low-bandwidth text and document files. The system reliably transmitted ASCII text and `.txt` documents at 200 bps with no frame loss under stable optical conditions.

The failure of the audio transmission phase is equally instructive: it precisely mapped the hard architectural boundary of software-defined GPIO bit-banging. Linux OS scheduling jitter is tolerable for asynchronous ASCII streams — a dropped bit is a single corrupted character. For Base64-encoded binary payloads, a single bit-slip desynchronises the entire decoding sequence, making software-only implementations fundamentally unsuitable for high-density binary transfer.

Key conclusions:

- Absolute-timestamp scheduling is essential for drift-free GPIO bit-banging on Linux.
- I/O batching provides disproportionate throughput gains in the 100–200 bps range.
- Base64 is a practical encoding bridge for binary payloads over ASCII protocols — but exposes jitter sensitivity in long transfers.
- Periodic `fsync()` is critical for fault-tolerant storage on embedded media.
- Collimation and rigid mechanical alignment are prerequisites for consistent optical operation beyond trivial distances.
- Scaling to audio and beyond requires hardware UART with analogue front-end conditioning (transimpedance amplifier, threshold comparator) or DMA-capable C drivers.